Empório da Música — Customer Support Agent

Technical Assessment Documentation

Candidate: Andréia Botto

Career: AI Engineer – Full-Stack para Aplicações com IA Generativa – Vaga afirmativa para mulheres cis e trans

Company: Artefact

Document: Project README & Documentation (English)

This PDF consolidates the project README and technical documentation for the AI Engineer challenge submission.

1. Overview

Python prototype of a messaging agent for the fictional store **Empório da Música** (Campo Grande/MS), developed as the Artefact AI Engineer technical challenge. The agent combines **tool calling** (operational data in SQLite) with **RAG** (store policies PDF) and uses a **free-tier LLM** via Groq.

1.1 Features

- Persona aligned with the store tone (informal–professional)
- Catalog, pricing, stock, and active promotions lookup
- Order status by `order_id`, tracking, email, or phone (minimized LGPD exposure)
- Policies: hours, address, payment, exchanges/returns, shipping, warranty, LGPD
- Store date/time tool (`get_current_datetime`) for “are we open now?”
- Polite refusal for out-of-scope requests (accessories), with a gate in the catalog tool
- Session conversation history with summarized tool facts
- Graphical interface with Streamlit
- Automated tests (unit + integration + evals tied to examples)
- CI (GitHub Actions) and local or Docker Compose execution

2. Architecture

```
User (CLI or Streamlit)
    → LangGraph ReAct (step limit + Groq retry)
        → Groq openai/gpt-oss-120b
        → Tools → SQLite (from CSVs)
        → RAG tool → Chroma + multilingual sentence-transformers (local)
        → get_current_datetime (America/Campo_Grande)
```

2.1 Stack

Component	Technology
Language	Python 3.11+
Agent	LangChain + LangGraph
LLM	Groq `openai/gpt-oss-120b` (free tier)
Embeddings	`paraphrase-multilingual-MiniLM-L12-v2` (local)
Vector store	ChromaDB (MMR)
Data	pandas + SQLite
CLI	Typer + Rich
UI	Streamlit
Tests	pytest (+ YAML evals)
Containers	Docker multi-stage + Compose

3. Prerequisites & Local Setup

- Free account at console.groq.com and an API key
- Python 3.11+ **or** Docker

3.1 Local installation

```
python -m venv .venv
```

```
# Windows PowerShell
.\.venv\Scripts\Activate.ps1

# Linux/macOS
source .venv/bin/activate

pip install -r requirements.txt
pip install -e .

cp .env.example .env
# edit .env and set GROQ_API_KEY=...
```

3.2 Ingestion and chat

```
python -m emporio_agent ingest
python -m emporio_agent chat
# or simply:
python -m emporio_agent
```

CLI chat commands: `/nova` (clear history), `/sair`, `/help`. Verbose mode (shows tool calls): `python -m emporio_agent chat --verbose`.

3.3 Streamlit UI

```
python -m emporio_agent ui
# custom port:
python -m emporio_agent ui --port 8501
```

Opens at http://localhost:8501: chat with suggestions, new conversation, agent restart, and tools toggle.

3.4 Docker setup

```
cp .env.example .env
# set GROQ_API_KEY

# Interactive CLI
docker compose up --build agent

# Streamlit UI
docker compose up --build ui

# Tests in container
docker compose run --rm test
```

The first run downloads the embedding model and indexes policies (may take a while). Volumes persist SQLite (`.data/`), Chroma (`.chroma/`), and the HuggingFace cache. The image runs as a non-root user.

4. Testing

```
# default: unit + integration (no live / no Groq network)
pytest

# unit only
pytest -m unit

# optional smoke with real API
pytest -m live
```

4.1 Test layers

Layer	What it proves	What it does **not** prove

Unit (tools/ingest/memory)	SQL, LGPD in the tool, compaction, prompt contracts	Real LLM tool choice
Integration (scripted LLM)	ReAct loop + stub tools **and** real SQLite tools	Persona / tool-choice of the Groq model
Evals (`tests/eval_cases.yaml`)	Core examples tied to asserts	Real embedding semantic quality (CI uses FakeEmbeddings for RAG)
Live	Greeting smoke with API	Full scenario coverage

CI (`.github/workflows/ci.yml`) runs `pytest -m "unit or integration"`.

5. Project Structure

```
src/emporio_agent/
  agent/          # prompts, LangGraph, memory
  data/           # CSV → SQLite and PDF → Chroma ingestion
  tools/          # products, orders, promotions, policies, datetime
  ui/             # Streamlit
  bootstrap.py    # shared CLI/UI init
  cli.py
data/             # CSVs and store policies PDF
examples/         # example conversations (illustrative, not captured transcripts)
tests/            # unit, integration, eval_cases.yaml
.github/workflows/ci.yml
Dockerfile
docker-compose.yml
```

6. Technical Decisions

- **Hybrid RAG + tools** — prices/stock/orders are structured (SQL is more reliable); policies are free text (RAG).
- **LangGraph ReAct** — explicit reasoning + tools loop, with `recursion_limit` and rate-limit retry.
- **Free Groq tier** — zero cost for the evaluator; good latency and tool-calling support. Future alternative: fully local Ollama.
- **Local multilingual embeddings** — better PT-BR fit without a second paid API; trade-off: larger image/cache.
- **SQLite** — zero infra; CSV fingerprint invalidates stale ingestion; `unit_price_brl` snapshot on items.
- **Compact memory** — removes raw tool traces (TPM) but keeps a `SystemMessage` with summarized facts.
- **LGPD in the tool** — identifiers with AND; exact phone; email/phone never in JSON; masked name without order_id/tracking.
- **Layered tests** — tools without LLM; agent with fake LLM; example-linked evals; optional live smoke.

7. Limitations and Next Steps

- Groq free-tier rate limits may require wait/retry (basic backoff is implemented)
- Product match is accent-folded substring (not full fuzzy/FTS)
- Order identification = possession of an identifier (not real auth / OTP)
- `unit_price_brl` is a catalog snapshot at ingestion — the items CSV has no real historical price
- RAG: section chunking + sub-chunk; policy CI uses FakeEmbeddings (semantic ranking is not a CI gate)
- Files in `examples/` are **expected scripts**, not verbatim CLI dumps
- No WhatsApp channel; online LLM-as-judge evals are a future evolution
- Streamlit UI has no access authentication (adequate for a local demo)

8. Interaction Examples

Core (start here): `06` LGPD, `08` security, `09` multi-turn, `17` ambiguous policy, `01` catalog.

Full list under `examples/`:

- Guitars up to R$ 1000
- Store address
- Takamine GD20 price
- Return for change of mind (policy + order)
- Out of scope (strings/pedals)
- LGPD — order without identifier
- Order status with a valid identifier
- Security — jailbreak / prompt injection
- Multi-turn context (session memory)
- Active promotions + non-stackable PIX
- Out of stock + alternatives
- Shipping and delivery
- Warranty
- Complaint with empathy
- Nonexistent product (honesty)
- Business hours / after hours
- Ambiguous policy → escalate to management

9. Use of Coding Assistants

This project was built with assistance from **Cursor** (coding agent): reading the challenge PDF, scaffolding the hybrid architecture, implementing tools/RAG/CLI/Docker/tests, and drafting the README.

Decisions made with the assistant (kept after human review):

- Separate structured facts (SQL) from policies (RAG), instead of embedding everything
- Groq free tier + local embeddings (zero cost for the evaluator)
- Compact history for TPM, retaining tool summaries instead of full traces
- Enforce LGPD in the tool (not only in the prompt)
- YAML evals linked to examples for what the challenge asks to judge

10. Data

Files under `data/` were provided by the challenge. Not every field needs to be used; the agent prioritizes what improves support quality (products, orders, customers, active promotions, and policies).

— End of documentation —